** Check models **

In [1]:
```
import requests
import os

api_key = I deleted my API key. Your API key will appear here.

headers = {
    "Authorization": f"Bearer {api_key}",
    "Content-Type": "application/json"
}

response = requests.get(url, headers=headers)

print(response.json())
```

```
{'object': 'list', 'data': [{'id': 'meta-llama/llama-prompt-guard-2-22m',
'object': 'model', 'created': 1748632101, 'owned_by': 'Meta', 'active': True,
'context_window': 512, 'public_apps': None, 'max_completion_tokens': 512,
'name': 'Llama Prompt Guard 2 22M', 'input_modalities': ['text'],
'output_modalities': ['text'], 'context_length': 512, 'max_output_length':
512, 'pricing': {'prompt': '0.00000003', 'completion': '0.00000003', 'image':
'0', 'request': '0', 'input_cache_read': '0.000000015'},
'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed',
'max_tokens']}, {'id': 'canopylabs/orpheus-v1-english', 'object': 'model',
'created': 1766186316, 'owned_by': 'Canopy Labs', 'active': True,
'context_window': 4000, 'public_apps': None, 'max_completion_tokens': 50000,
'name': 'Canopy Labs Orpheus V1 English', 'input_modalities': ['text'],
'output_modalities': ['speech'], 'context_length': 4000, 'max_output_length':
50000, 'pricing': {'prompt': '0.000022', 'image': '0', 'request': '0',
'input_cache_read': '0.000011'}}, {'id': 'llama-3.3-70b-versatile', 'object':
'model', 'created': 1733447754, 'owned_by': 'Meta', 'active': True,
'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 32768,
'hugging_face_id': 'meta-llama/Llama-3.3-70B-Instruct', 'name': 'Llama 3.3
70B', 'input_modalities': ['text'], 'output_modalities': ['text'],
'context_length': 131072, 'max_output_length': 32768, 'pricing': {'prompt':
'0.00000059', 'completion': '0.00000079', 'image': '0', 'request': '0',
'input_cache_read': '0.000000295'}, 'supported_sampling_parameters':
['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features':
['tools', 'json_mode']}, {'id': 'meta-llama/llama-prompt-guard-2-86m',
'object': 'model', 'created': 1748632165, 'owned_by': 'Meta', 'active': True,
'context_window': 512, 'public_apps': None, 'max_completion_tokens': 512,
'name': 'Prompt Guard 2 86M', 'input_modalities': ['text'],
'output_modalities': ['text'], 'context_length': 512, 'max_output_length':
512, 'pricing': {'prompt': '0.00000004', 'completion': '0.00000004', 'image':
'0', 'request': '0', 'input_cache_read': '0.00000002'},
'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed',
'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'openai/gpt-oss-
20b', 'object': 'model', 'created': 1754407957, 'owned_by': 'OpenAI',
'active': True, 'context_window': 131072, 'public_apps': None,
'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-20b',
'name': 'GPT OSS 20B', 'input_modalities': ['text'], 'output_modalities':
['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing':
{'prompt': '0.000000075', 'completion': '0.0000003', 'image': '0', 'request':
'0', 'input_cache_read': '0.0000000375'}, 'supported_sampling_parameters':
['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features':
['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id':
'openai/gpt-oss-120b', 'object': 'model', 'created': 1754408224, 'owned_by':
'OpenAI', 'active': True, 'context_window': 131072, 'public_apps': None,
'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-120b',
'name': 'GPT OSS 120B', 'input_modalities': ['text'], 'output_modalities':
```

['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing': {'prompt': '0.00000015', 'completion': '0.0000006', 'image': '0', 'request': '0', 'input_cache_read': '0.000000075'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id': 'qwen/qwen3.6-27b', 'object': 'model', 'created': 1778288776, 'owned_by': 'Alibaba Cloud', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 16384, 'hugging_face_id': 'Qwen/Qwen3.6-27B', 'name': 'Qwen/Qwen3.6-27B', 'input_modalities': ['text', 'image'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 16384, 'pricing': {'prompt': '0.0000006', 'completion': '0.000003', 'image': '0', 'request': '0', 'input_cache_read': '0.0000003'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'reasoning']}, {'id': 'llama-3.1-8b-instant', 'object': 'model', 'created': 1693721698, 'owned_by': 'Meta', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 131072, 'hugging_face_id': 'meta-llama/Llama-3.1-8B-Instruct', 'name': 'Llama 3.1 8B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 131072, 'pricing': {'prompt': '0.00000005', 'completion': '0.00000008', 'image': '0', 'request': '0', 'input_cache_read': '0.000000025'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode']}, {'id': 'whisper-large-v3', 'object': 'model', 'created': 1693721698, 'owned_by': 'OpenAI', 'active': True, 'context_window': 448, 'public_apps': None, 'max_completion_tokens': 448, 'hugging_face_id': 'openai/whisper-large-v3', 'name': 'Whisper', 'input_modalities': ['audio'], 'output_modalities': ['transcription'], 'context_length': 448, 'max_output_length': 448}, {'id': 'whisper-large-v3-turbo', 'object': 'model', 'created': 1728413088, 'owned_by': 'OpenAI', 'active': True, 'context_window': 448, 'public_apps': None, 'max_completion_tokens': 448, 'hugging_face_id': 'openai/whisper-large-v3-turbo', 'name': 'Whisper Large V3 Turbo', 'input_modalities': ['audio'], 'output_modalities': ['transcription'], 'context_length': 448, 'max_output_length': 448}, {'id': 'openai/gpt-oss-safeguard-20b', 'object': 'model', 'created': 1761708789, 'owned_by': 'OpenAI', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 65536, 'hugging_face_id': 'openai/gpt-oss-safeguard-20b', 'name': 'Safety GPT OSS 20B', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 65536, 'pricing': {'prompt': '0.000000075', 'completion': '0.0000003', 'image': '0', 'request': '0', 'input_cache_read': '0.0000000375'}, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['tools', 'json_mode', 'structured_outputs', 'reasoning']}, {'id': 'groq/compound-mini', 'object': 'model', 'created': 1756949707, 'owned_by': 'Groq', 'active': True, 'context_window': 131072, 'public_apps': None, 'max_completion_tokens': 8192, 'hugging_face_id': 'groq/compound-beta-mini', 'name': 'Compound Mini', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 131072, 'max_output_length': 8192, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id': 'canopylabs/orpheus-arabic-saudi', 'object': 'model', 'created': 1765926439, 'owned_by': 'Canopy Labs', 'active': True, 'context_window': 4000, 'public_apps': None, 'max_completion_tokens': 50000, 'name': 'Canopy Labs Orpheus Arabic Saudi', 'input_modalities': ['text'], 'output_modalities': ['speech'], 'context_length': 4000, 'max_output_length': 50000, 'pricing': {'prompt': '0.00004', 'image': '0', 'request': '0', 'input_cache_read': '0.00002'}}, {'id': 'allam-2-7b', 'object': 'model', 'created': 1737672203, 'owned_by': 'SDAIA', 'active': True, 'context_window': 4096, 'public_apps': None, 'max_completion_tokens': 4096, 'hugging_face_id': 'ALLaM-AI/ALLaM-2.0-7B-Instruct', 'name': 'ALLaM-2-7b', 'input_modalities': ['text'], 'output_modalities': ['text'], 'context_length': 4096, 'max_output_length':

```
4096, 'supported_sampling_parameters': ['temperature', 'top_p', 'stop',
'seed', 'max_tokens'], 'supported_features': ['json_mode']}, {'id':
'groq/compound', 'object': 'model', 'created': 1756949530, 'owned_by': 'Groq',
'active': True, 'context_window': 131072, 'public_apps': None,
'max_completion_tokens': 8192, 'hugging_face_id': 'groq/compound-beta',
'name': 'Compound', 'input_modalities': ['text'], 'output_modalities':
['text'], 'context_length': 131072, 'max_output_length': 8192,
'supported_sampling_parameters': ['temperature', 'top_p', 'stop', 'seed',
'max_tokens'], 'supported_features': ['json_mode']}]}
```

** Canvas 1 **

In [2]:

```python
# ========================================================
# IMPORTS
# ========================================================

import json
import asyncio
import os
import pandas as pd

from groq import AsyncGroq

from bert_score import score
from rouge_score import rouge_scorer


# ========================================================
# CONFIG
# ========================================================

# Khuyến nghị:
# export GROQ_API_KEY="gsk_xxxxxxxxxxxxxxxxxxxxxxxx"

GROQ_API_KEY = I deleted my API key, Input your API key here

# Hoặc để thử nhanh:
# GROQ_API_KEY = "gsk_xxxxxxxxxxxxxxxxxxxxxxxx"

if not GROQ_API_KEY:
    raise RuntimeError(
        "Environment variable GROQ_API_KEY not found."
    )


# ========================================================
# MODEL
# ========================================================

MODEL_NAME = "llama-3.1-8b-instant"
nmodel = "Llama-3.1-8B"

# ========================================================
# FILE PATHS
# ========================================================

TEST_FILE = "./Training_Data/test.json"

CHECKPOINT_CSV = (
    "./Training_Data/"
    f"groq_{nmodel}_checkpoint.csv"
```

```
)

OUTPUT_CSV = (
    "./Training_Data/"
    f"Baseline_Groq_{nmodel}_predictions.csv"
)


# ============================================================
# REQUEST SETTINGS
# ============================================================

# Chỉ gửi request tiếp theo sau khi
# đã nhận được response hoàn chỉnh.

DELAY_AFTER_RESPONSE = 4.5

# Retry tối đa với các lỗi tạm thời
MAX_RETRY = 1000

# Thời gian chờ tối đa giữa các lần retry
MAX_WAIT_SECONDS = 60


# ============================================================
# CREATE GROQ CLIENT
# ============================================================

client = AsyncGroq(
    api_key=GROQ_API_KEY
)


print("=" * 60)
print("Groq Benchmark")
print("=" * 60)
print("Model      :", MODEL_NAME)
print("Checkpoint :", CHECKPOINT_CSV)
print("Output     :", OUTPUT_CSV)
print("=" * 60)
```

```
/opt/anaconda3/envs/LLM/lib/python3.11/site-packages/tqdm/auto.py:21:
TqdmWarning: IProgress not found. Please update jupyter and ipywidgets. See
https://ipywidgets.readthedocs.io/en/stable/user_install.html
  from .autonotebook import tqdm as notebook_tqdm
============================================================
Groq Benchmark
============================================================
Model      : llama-3.1-8b-instant
Checkpoint : ./Training_Data/groq_Llama-3.1-8B_checkpoint.csv
Output     : ./Training_Data/Baseline_Groq_Llama-3.1-8B_predictions.csv
============================================================
```

** Canvas 2 **

In [3]:

```
# ============================================================
# LOAD DATASET
# ============================================================

dataset = []

print("\nLoading dataset...")
```

```
with open(TEST_FILE, "r", encoding="utf-8") as f:

    for line in f:

        line = line.strip()

        if line:

            dataset.append(
                json.loads(line)
            )


TOTAL_SAMPLES = len(dataset)

print(
    f"Loaded {TOTAL_SAMPLES} samples."
)


# ================================================
# LOAD CHECKPOINT
# ================================================

def load_checkpoint():

    predictions = []

    references = []

    start_idx = 0


    if not os.path.exists(CHECKPOINT_CSV):

        print(
            "\nNo checkpoint found."
        )

        print(
            "Start from sample 1."
        )

        return (
            predictions,
            references,
            start_idx
        )


    print(
        "\nCheckpoint detected."
    )


    try:

        checkpoint = pd.read_csv(
            CHECKPOINT_CSV,
            encoding="utf-8"
```

```
        )


        if "prediction" not in checkpoint.columns:

            raise RuntimeError(
                "Prediction column not found."
            )


        if "ground_truth" not in checkpoint.columns:

            raise RuntimeError(
                "Ground truth column not found."
            )


        checkpoint["prediction"] = (
            checkpoint["prediction"]
            .fillna("")
        )


        checkpoint["ground_truth"] = (
            checkpoint["ground_truth"]
            .fillna("")
        )


        predictions = (
            checkpoint["prediction"]
            .tolist()
        )


        references = (
            checkpoint["ground_truth"]
            .tolist()
        )


        start_idx = len(
            predictions
        )


        if start_idx > TOTAL_SAMPLES:

            raise RuntimeError(
                "Checkpoint larger than dataset."
            )


        print(
            f"Recovered "
            f"{start_idx} completed samples."
        )


        if start_idx < TOTAL_SAMPLES:
```

```
            print(
                f"Resume from sample "
                f"{start_idx + 1}/{TOTAL_SAMPLES}"
            )

        else:

            print(
                "All samples already completed."
            )


        return (
            predictions,
            references,
            start_idx
        )


    except Exception as e:

        print(
            "\nFailed to read checkpoint."
        )

        print(e)

        print(
            "\nRestart from beginning."
        )

        return (
            [],
            [],
            0
        )


# ====================================================
# SAVE CHECKPOINT
# ====================================================

def save_checkpoint(
    predictions,
    references
):

    completed = len(
        predictions
    )


    checkpoint = pd.DataFrame(
        {

            "instruction":
            [
                x["instruction"]
                for x in dataset[:completed]
            ],
```

```
            "input":
            [
                x["input"]
                for x in dataset[:completed]
            ],

            "ground_truth":
            references,

            "prediction":
            predictions

        }
    )


    checkpoint.to_csv(
        CHECKPOINT_CSV,
        index=False,
        encoding="utf-8"
    )


    print(
        f"💾 Checkpoint saved "
        f"({completed}/{TOTAL_SAMPLES})"
    )
```

```
Loading dataset...
Loaded 121 samples.
```

** Canvas 3 **

In [4]:

```
# ====================================================
# SEND REQUEST TO GROQ
# NEVER SKIP A SAMPLE
# ====================================================

async def send_and_wait_for_response(
        sample,
        idx,
        total
):

    prompt = f"""
Instruction:
{sample['instruction']}

Input:
{sample['input']}

Answer:
"""

    attempt = 1

    while True:

        try:

            print(
                f"\n[{idx+1}/{total}] "
```

```
            f"Sending request to Groq "
            f"(attempt {attempt})"
        )


        completion = await client.chat.completions.create(

            model=MODEL_NAME,

            messages=[
                {
                    "role": "user",
                    "content": prompt
                }
            ],

            temperature=0.0,

            max_completion_tokens=2048,

            top_p=1.0,

            stream=False

        )


        if (
            completion is None
            or len(completion.choices) == 0
        ):
            raise RuntimeError(
                "Empty response from Groq."
            )


        prediction = (
            completion
            .choices[0]
            .message
            .content
        )


        if prediction is None:

            raise RuntimeError(
                "Groq returned None."
            )


        prediction = prediction.strip()


        if prediction == "":

            raise RuntimeError(
                "Groq returned empty text."
            )
```

```
            print("✅ Response received")

            preview = prediction.replace(
                "\n",
                " "
            )[:120]

            print(preview + "...")

            return prediction


        except Exception as e:

            error_message = str(e)

            print(
                "\n❌ Groq Error:"
            )

            print(error_message)


            # ==========================================
            # QUOTA EXCEEDED -> STOP PROGRAM
            # ==========================================

            if (
                "429" in error_message
                or "rate_limit" in error_message.lower()
                or "quota" in error_message.lower()
                or "resource_exhausted" in error_message.lower()
            ):

                print("\n" + "=" * 60)
                print("❌ QUOTA EXCEEDED")
                print("Program stopped.")
                print("Checkpoint is preserved.")
                print("Run again after quota resets.")
                print("=" * 60)

                raise SystemExit(
                    "Stopped because quota was exceeded."
                )


            # ==========================================
            # OTHER ERRORS -> RETRY
            # ==========================================

            wait_time = min(
                MAX_WAIT_SECONDS,
                attempt * 5
            )

            print(
                f"Retry after {wait_time} seconds..."
            )
```

```
            await asyncio.sleep(
                wait_time
            )

            attempt += 1
```

In [5]:

```
# ======================================================
# EVALUATION
# ======================================================

async def evaluate_and_save(
        predictions,
        references
):

    print(
        "\n========================================="
    )

    print(
        "Computing evaluation metrics..."
    )


    assert len(predictions) == len(references)

    assert len(predictions) == TOTAL_SAMPLES


    # ================================================
    # BERTScore
    # ================================================

    print("\nComputing BERTScore...")

    P, R, F1 = score(

        predictions,

        references,

        lang="en",

        verbose=True

    )


    # ================================================
    # ROUGE-L
    # ================================================

    print("\nComputing ROUGE-L...")


    scorer = rouge_scorer.RougeScorer(

        ["rougeL"],

        use_stemmer=True

    )
```

```
rouge_precision = []
rouge_recall = []
rouge_f1 = []

for pred, ref in zip(
    predictions,
    references
):

    result = scorer.score(
        ref,
        pred
    )

    rouge = result["rougeL"]

    rouge_precision.append(
        rouge.precision
    )

    rouge_recall.append(
        rouge.recall
    )

    rouge_f1.append(
        rouge.fmeasure
    )

# ================================================
# SAVE RESULT
# ================================================

print("\nSaving CSV...")

final_df = pd.DataFrame(
    {
```

```
            "instruction":
            [
                x["instruction"]
                for x in dataset
            ],
            "input":
            [
                x["input"]
                for x in dataset
            ],
            "ground_truth":
            references,
            "prediction":
            predictions,
            "bert_precision":
            P.cpu().numpy(),
            "bert_recall":
            R.cpu().numpy(),
            "bert_f1":
            F1.cpu().numpy(),
            "rougeL_precision":
            rouge_precision,
            "rougeL_recall":
            rouge_recall,
            "rougeL_f1":
            rouge_f1
```

```
        }
    )

    final_df.to_csv(
        OUTPUT_CSV,
        index=False,
        encoding="utf-8"
    )

    # ================================================
    # REPORT
    # ================================================
    print("\n========================================")
    print("Average BERTScore")
    print("----------------------------------------")
    print(
        f"Precision : {P.mean().item():.4f}"
    )
    print(
        f"Recall    : {R.mean().item():.4f}"
    )
    print(
        f"F1        : {F1.mean().item():.4f}"
    )

    print("\n========================================")
    print("Average ROUGE-L")
    print("----------------------------------------")
    print(
        f"Precision : "
        f"{sum(rouge_precision)/len(rouge_precision):.4f}"
    )
```

```
    print(
        f"Recall    : "
        f"{sum(rouge_recall)/len(rouge_recall):.4f}"
    )

    print(
        f"F1        : "
        f"{sum(rouge_f1)/len(rouge_f1):.4f}"
    )

    print("\n========================================")
    print("Evaluation completed.")
    print(f"Output CSV : {OUTPUT_CSV}")
    print("========================================")
```

** Canvas 4 **

In [ ]:

```
# ======================================================
# MAIN LOOP
# ======================================================

async def main():

    # ----------------------------------------------
    # LOAD CHECKPOINT
    # ----------------------------------------------

    (
        predictions,
        references,
        start_idx
    ) = load_checkpoint()


    # ----------------------------------------------
    # ALL SAMPLES COMPLETED
    # ----------------------------------------------

    if start_idx >= TOTAL_SAMPLES:

        print("\nAll samples already completed.")
        print("Start evaluation...\n")

        await evaluate_and_save(
            predictions,
            references
        )

        return
```

```
print("\n====================================")
print("Starting inference...")
print("====================================")


# ----------------------------------------
# INFERENCE LOOP
# ----------------------------------------

for idx in range(
    start_idx,
    TOTAL_SAMPLES
):

    sample = dataset[idx]


    # ----------------------------------------
    # WAIT UNTIL RESPONSE IS RECEIVED
    # ----------------------------------------

    prediction = await send_and_wait_for_response(

        sample=sample,

        idx=idx,

        total=TOTAL_SAMPLES

    )


    # ----------------------------------------
    # SAVE RESULT IN MEMORY
    # ----------------------------------------

    predictions.append(
        prediction
    )

    references.append(
        sample["output"]
    )


    print(
        f"Completed "
        f"{len(predictions)}/{TOTAL_SAMPLES}"
    )


    # ----------------------------------------
    # SAVE CHECKPOINT
    # ----------------------------------------

    if (
        len(predictions) % 2 == 0
        or idx == TOTAL_SAMPLES - 1
    ):

        save_checkpoint(
```

```
                predictions,
                references
            )


        # ----------------------------------------
        # WAIT BEFORE NEXT REQUEST
        # ----------------------------------------

        if idx < TOTAL_SAMPLES - 1:

            print(
                f"Waiting "
                f"{DELAY_AFTER_RESPONSE}s..."
            )

            await asyncio.sleep(
                DELAY_AFTER_RESPONSE
            )


    # ----------------------------------------
    # ALL RESPONSES RECEIVED
    # ----------------------------------------

    print("\n========================================")
    print("All Groq responses completed.")
    print("Start evaluation...")
    print("========================================\n")


    await evaluate_and_save(

        predictions,

        references

    )
```

** Canvas 5 **

In [ ]:

** Canvas 6 **

In [7]:
```
try:

    await main()

except KeyboardInterrupt:

    print("\nProgram interrupted by user.")
    print("Checkpoint has been preserved.")

except SystemExit as e:

    print(e)

except Exception as e:

    print(type(e).__name__)
    print(e)
```

```
    raise
```

```
No checkpoint found.
Start from sample 1.

========================================
Starting inference...
========================================

[1/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the competitive position of itemset A [39, 48, 10486] compared to
itemset B [39, 48, 6738], we will evaluate ...
Completed 1/121
Waiting 4.5s...

[2/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 4444] satisfies the frequent itemset (FI)
criteria, we need to check if its support is ...
Completed 2/121
Waiting 4.5s...

[3/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will analyze the
given metrics:  1. **Number of Supports**: It...
Completed 3/121
Waiting 4.5s...

[4/121] Sending request to Groq (attempt 1)
✅ Response received
To determine which itemset has high performance compared to the other, we need
to analyze the key metrics provided for b...
Completed 4/121
Waiting 4.5s...

[5/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [48, 7560] is a frequent itemset (FI), we need
to compare its support value with the su...
Completed 5/121
💾 Checkpoint saved (5/121)
Waiting 4.5s...

[6/121] Sending request to Groq (attempt 1)
✅ Response received
Based on the provided metrics, we can analyze the competitive position of
itemset [14551] compared to itemset [1398]. He...
Completed 6/121
Waiting 4.5s...

[7/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset [39, 41, 5156] compared to
itemset [48, 41, 169], we will analyze th...
Completed 7/121
Waiting 4.5s...
```

```
[8/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will analyze the
given metrics:  1. **Number of Supports**: Th...
Completed 8/121
Waiting 4.5s...

[9/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [39, 5520], we will examine the given parameters:  1.
**Number of Supports**: The itemset [39, 55...
Completed 9/121
Waiting 4.5s...

[10/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [9652] satisfies the frequent itemset (FI)
criteria, we need to check if its support is grea...
Completed 10/121
💾 Checkpoint saved (10/121)
Waiting 4.5s...

[11/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset [39, 6268] compared to
itemset [39, 14031], we will analyze their ch...
Completed 11/121
Waiting 4.5s...

[12/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [41, 100], we will examine the given parameters:  1.
**Number of Supports**: The itemset [41, 100...
Completed 12/121
Waiting 4.5s...

[13/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we need to analyze the
given metrics. Here's a comparison of the ...
Completed 13/121
Waiting 4.5s...

[14/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. **Number of supports**: This is the num...
Completed 14/121
Waiting 4.5s...

[15/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [2409] with itemset [13050], we need
to analyze the given metrics and determin...
Completed 15/121
💾 Checkpoint saved (15/121)
```

```
Waiting 4.5s...

[16/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 16176] satisfies the frequent itemset (FI)
criteria, we need to check if its support is...
Completed 16/121
Waiting 4.5s...

[17/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze
their characteristics and how they relate ...
Completed 17/121
Waiting 4.5s...

[18/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [48, 2389] qualifies as a frequent itemset (FI),
let's analyze its metrics:  1. **Support**...
Completed 18/121
Waiting 4.5s...

[19/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. **Number of supports**: This is the num...
Completed 19/121
Waiting 4.5s...

[20/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [41, 1493], we will examine the given parameters and
determine whether it meets the criteria for ...
Completed 20/121
💾 Checkpoint saved (20/121)
Waiting 4.5s...

[21/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [1598] with itemset [7160], we will
analyze the given metrics:  1. **Number of Sup...
Completed 21/121
Waiting 4.5s...

[22/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [9026] with itemset [6554], we need
to analyze the given metrics and identify ...
Completed 22/121
Waiting 4.5s...

[23/121] Sending request to Groq (attempt 1)
✅ Response received
The itemset [48, 5853] fails to achieve strong performance due to its low
support ratio of 0.00013611306458564916. This ...
Completed 23/121
```

```
Waiting 4.5s...

[24/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [7646], we will examine the given parameters and
determine whether it meets the criteria for bein...
Completed 24/121
Waiting 4.5s...

[25/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we need to analyze the
given metrics. Here's a comparison of the ...
Completed 25/121
💾 Checkpoint saved (25/121)
Waiting 4.5s...

[26/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze
the given metrics and identify the key dif...
Completed 26/121
Waiting 4.5s...

[27/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 5086], we will examine the given parameters and
determine whether it meets the criteria for ...
Completed 27/121
Waiting 4.5s...

[28/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset A compared to itemset B, we
will analyze the given metrics.  **Stren...
Completed 28/121
Waiting 4.5s...

[29/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [1043] with itemset [1213], we will
analyze the given metrics:  1. **Number of Sup...
Completed 29/121
Waiting 4.5s...

[30/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [39, 6891], we will examine the given parameters and
determine whether it meets the criteria for ...
Completed 30/121
💾 Checkpoint saved (30/121)
Waiting 4.5s...

[31/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 6949] satisfies the frequent itemset (FI)
criteria, we need to check if its support is ...
```

```
Completed 31/121
Waiting 4.5s...

[32/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [1412], we will examine the given parameters and
determine whether it meets the criteria for bein...
Completed 32/121
Waiting 4.5s...

[33/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will evaluate them
based on the following metrics:  1. **Numbe...
Completed 33/121
Waiting 4.5s...

[34/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 9066], we will examine the given parameters and
determine whether it meets the criteria for ...
Completed 34/121
Waiting 4.5s...

[35/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [39, 48, 8253] qualifies as a frequent itemset
(FI), let's analyze its metrics:  1. **Suppo...
Completed 35/121
💾 Checkpoint saved (35/121)
Waiting 4.5s...

[36/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will analyze the
given metrics:  1. **Number of Supports**: It...
Completed 36/121
Waiting 4.5s...

[37/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will analyze the
given metrics:  1. **Number of Supports**: Th...
Completed 37/121
Waiting 4.5s...

[38/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. **Number of Supports**: This is the num...
Completed 38/121
Waiting 4.5s...

[39/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 41, 869] and determine whether it meets the
criteria for being a frequent itemset (FI), we w...
```

```
Completed 39/121
Waiting 4.5s...

[40/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 48, 388] is a frequent itemset (FI), we need
to compare its support value with the supp...
Completed 40/121
💾 Checkpoint saved (40/121)
Waiting 4.5s...

[41/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [41, 1444] is a frequent itemset (FI), we need
to compare its support value with the su...
Completed 41/121
Waiting 4.5s...

[42/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [13288] with itemset [879], we will
evaluate the following metrics:  1. **Number o...
Completed 42/121
Waiting 4.5s...

[43/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 15459], we will examine the given parameters and
determine whether it meets the criteria for...
Completed 43/121
Waiting 4.5s...

[44/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [14916] is a frequent itemset (FI), we need to
compare its support value with the suppo...
Completed 44/121
Waiting 4.5s...

[45/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [11857] with itemset [2784], we need
to analyze the given metrics and determin...
Completed 45/121
💾 Checkpoint saved (45/121)
Waiting 4.5s...

[46/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [39, 2385] with itemset [48, 6360], we
will analyze the following metrics:  1. **N...
Completed 46/121
Waiting 4.5s...

[47/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the relative performance of itemset [41, 6598] against itemset [41,
```

1313], we need to compare their various m...
Completed 47/121
Waiting 4.5s...

[48/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will evaluate them based on the following metrics: 1. **Numbe...
Completed 48/121
Waiting 4.5s...

[49/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [39, 16178], we will examine the given parameters and determine whether it meets the criteria for...
Completed 49/121
Waiting 4.5s...

[50/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [48, 41, 318] satisfies the frequent itemset (FI) criteria, we need to check if its support ...
Completed 50/121
💾 Checkpoint saved (50/121)
Waiting 4.5s...

[51/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [41, 4515] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Support**...
Completed 51/121
Waiting 4.5s...

[52/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [14870] with itemset [10663], we will analyze the given data points. **Number of ...
Completed 52/121
Waiting 4.5s...

[53/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [48, 6418] satisfies the frequent itemset (FI) criteria, we need to check if its support is ...
Completed 53/121
Waiting 4.5s...

[54/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze the given metrics and identify the key dif...
Completed 54/121
Waiting 4.5s...

[55/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why itemset [39, 498] has high performance compared to itemset

```
[48, 5412] based on key metrics, let's analy...
Completed 55/121
💾 Checkpoint saved (55/121)
Waiting 4.5s...

[56/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 5538] satisfies the frequent itemset (FI)
criteria, we need to check if its support is ...
Completed 56/121
Waiting 4.5s...

[57/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 41, 5962] is a frequent itemset (FI), we need
to compare its support value with the sup...
Completed 57/121
Waiting 4.5s...

[58/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze
the given metrics and identify the key dif...
Completed 58/121
Waiting 4.5s...

[59/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the relative performance of itemset A [39, 10576] against itemset B
[39, 2026], we need to compare their perf...
Completed 59/121
Waiting 4.5s...

[60/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will evaluate them
based on the following metrics:  1. **Numbe...
Completed 60/121
💾 Checkpoint saved (60/121)
Waiting 4.5s...

[61/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the given itemset [39, 41, 2257], we will evaluate it based on the
provided criteria: number of supports, ave...
Completed 61/121
Waiting 4.5s...

[62/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the relative performance of itemset A and itemset B, we need to
compare their characteristics. Here's a compa...
Completed 62/121
Waiting 4.5s...

[63/121] Sending request to Groq (attempt 1)
✅ Response received
```

To compare the business impact of itemset [7694] and itemset [9277], we need to analyze their characteristics and how th...
Completed 63/121
Waiting 4.5s...

[64/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [41, 3746] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Support**...
Completed 64/121
Waiting 4.5s...

[65/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the relative performance of itemset A and itemset B, we need to compare their characteristics. Here's a compa...
Completed 65/121
💾 Checkpoint saved (65/121)
Waiting 4.5s...

[66/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics: 1. Number of supports: This is a simple co...
Completed 66/121
Waiting 4.5s...

[67/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics: 1. **Number of Supports**: This is a simpl...
Completed 67/121
Waiting 4.5s...

[68/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [41, 1739] with itemset [41, 4816], we will analyze the following metrics: 1. **N...
Completed 68/121
Waiting 4.5s...

[69/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [41, 1884], we will examine the given parameters and determine whether it meets the criteria for ...
Completed 69/121
Waiting 4.5s...

[70/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [48, 41, 5543] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Suppo...
Completed 70/121
💾 Checkpoint saved (70/121)
Waiting 4.5s...

[71/121] Sending request to Groq (attempt 1)

✅ Response received
To determine why the itemset [39, 48, 14056] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Supp...
Completed 71/121
Waiting 4.5s...

[72/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the competitive position of itemset [5442] compared to itemset [4491], we will evaluate them based on key met...
Completed 72/121
Waiting 4.5s...

[73/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [41, 6105] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Support**...
Completed 73/121
Waiting 4.5s...

[74/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [48, 41, 5886] satisfies the frequent itemset (FI) criteria, we need to check if its support...
Completed 74/121
Waiting 4.5s...

[75/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset [9998] compared to itemset [7538], we will analyze their respective ...
Completed 75/121
💾 Checkpoint saved (75/121)
Waiting 4.5s...

[76/121] Sending request to Groq (attempt 1)
✅ Response received
To determine which itemset has high performance compared to the other, we need to analyze the key metrics provided for b...
Completed 76/121
Waiting 4.5s...

[77/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why itemset [11083] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Support**: The s...
Completed 77/121
Waiting 4.5s...

[78/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [39, 7403] is a frequent itemset (FI), we need to compare its support value with the su...
Completed 78/121
Waiting 4.5s...

[79/121] Sending request to Groq (attempt 1)

✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. **Number of Supports**: This is a simpl...
Completed 79/121
Waiting 4.5s...

[80/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why itemset [41, 7991] has high performance compared to itemset
[39, 9564] based on key metrics, let's anal...
Completed 80/121
💾 Checkpoint saved (80/121)
Waiting 4.5s...

[81/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [4976] satisfies the frequent itemset (FI)
criteria, we need to check if its support is grea...
Completed 81/121
Waiting 4.5s...

[82/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [2221] satisfies the frequent itemset (FI)
criteria, we need to check if its support is grea...
Completed 82/121
Waiting 4.5s...

[83/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. **Number of supports**: This is the num...
Completed 83/121
Waiting 4.5s...

[84/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [41, 177] qualifies as a frequent itemset (FI),
let's analyze its metrics: 1. **Support**:...
Completed 84/121
Waiting 4.5s...

[85/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will analyze the
given metrics: 1. **Number of Supports**: It...
Completed 85/121
💾 Checkpoint saved (85/121)
Waiting 4.5s...

[86/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset [48, 1806] compared to
itemset [39, 11317], we will analyze the give...
Completed 86/121
Waiting 4.5s...

[87/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [41, 5025] with itemset [48, 11660], we will analyze the following metrics: 1. **...
Completed 87/121
Waiting 4.5s...

[88/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [48, 6974] and itemset [39, 3848], we need to analyze the given metrics and id...
Completed 88/121
Waiting 4.5s...

[89/121] Sending request to Groq (attempt 1)
✅ Response received
To evaluate the strengths and weaknesses of itemset [48, 9852] compared to itemset [39, 4802], we will analyze the given...
Completed 89/121
Waiting 4.5s...

[90/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [39, 1954] is a frequent itemset (FI), we need to compare its support value with the su...
Completed 90/121
💾 Checkpoint saved (90/121)
Waiting 4.5s...

[91/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [1213] satisfies the frequent itemset (FI) criteria, we need to check if its support is grea...
Completed 91/121
Waiting 4.5s...

[92/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [41, 2622] is a frequent itemset (FI), we need to compare its support value with the su...
Completed 92/121
Waiting 4.5s...

[93/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [48, 11397] is a frequent itemset (FI), we need to compare its support value with the s...
Completed 93/121
Waiting 4.5s...

[94/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why itemset [3159] qualifies as a frequent itemset (FI), let's analyze its metrics: 1. **Support**: The su...
Completed 94/121
Waiting 4.5s...

```
[95/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 5108], we will examine the given parameters and
determine whether it meets the criteria for ...
Completed 95/121
💾 Checkpoint saved (95/121)
Waiting 4.5s...

[96/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [48, 12202], we will examine the given parameters and
determine whether it meets the criteria for...
Completed 96/121
Waiting 4.5s...

[97/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [39, 41, 6303] (Itemset A) with
itemset [39, 41, 1232] (Itemset B), we need to...
Completed 97/121
Waiting 4.5s...

[98/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [6313], we will examine the given parameters and
determine whether it meets the criteria for bein...
Completed 98/121
Waiting 4.5s...

[99/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [48, 41, 8840] qualifies as a frequent itemset
(FI), let's analyze its metrics:  1. **Suppo...
Completed 99/121
Waiting 4.5s...

[100/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [48, 41, 5154] satisfies the frequent itemset (FI)
criteria, we need to check if its support...
Completed 100/121
💾 Checkpoint saved (100/121)
Waiting 4.5s...

[101/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we need to analyze their performance based
on the given metrics. Here's a comparison...
Completed 101/121
Waiting 4.5s...

[102/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [39, 1274] and itemset [39, 9235], we
need to analyze the given metrics. Here's a ...
Completed 102/121
Waiting 4.5s...
```

```
[103/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [39, 48, 4385] qualifies as a frequent itemset
(FI), let's analyze its metrics:  1. **Suppo...
Completed 103/121
Waiting 4.5s...

[104/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 2187] satisfies the frequent itemset (FI)
criteria, we need to check if its support is ...
Completed 104/121
Waiting 4.5s...

[105/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset [48, 14024] and itemset [39, 11546],
we need to analyze the given metrics and ...
Completed 105/121
💾 Checkpoint saved (105/121)
Waiting 4.5s...

[106/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset [2486] and itemset [2855], we will analyze the following
metrics:  1. **Number of Supports**: This is...
Completed 106/121
Waiting 4.5s...

[107/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze
their characteristics and how they relate ...
Completed 107/121
Waiting 4.5s...

[108/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether itemset [4417] is a frequent itemset (FI), we need to
compare its support value with the support th...
Completed 108/121
Waiting 4.5s...

[109/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [3259] is a frequent itemset (FI), we need to
compare its support value with the suppor...
Completed 109/121
Waiting 4.5s...

[110/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [39, 48, 1807], we will examine the given metrics:  1.
**Number of Supports**: The itemset has 47...
Completed 110/121
💾 Checkpoint saved (110/121)
```

```
Waiting 4.5s...

[111/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the business impact of itemset A and itemset B, we need to analyze
the given metrics and identify the key dif...
Completed 111/121
Waiting 4.5s...

[112/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset A and itemset B, we will calculate the following metrics:
1. Number of supports: This is the number ...
Completed 112/121
Waiting 4.5s...

[113/121] Sending request to Groq (attempt 1)
✅ Response received
To determine whether the itemset [39, 48, 306] is a frequent itemset (FI), we
need to compare its support value with the...
Completed 113/121
Waiting 4.5s...

[114/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [6447] with itemset [2490], we will
analyze the given data points.  **Number of Su...
Completed 114/121
Waiting 4.5s...

[115/121] Sending request to Groq (attempt 1)
✅ Response received
To analyze the itemset [604], we will examine the given parameters and
determine whether it meets the criteria for being...
Completed 115/121
💾 Checkpoint saved (115/121)
Waiting 4.5s...

[116/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset A and itemset B, we will evaluate them
based on the following metrics:  1. **Numbe...
Completed 116/121
Waiting 4.5s...

[117/121] Sending request to Groq (attempt 1)
✅ Response received
To compare itemset [9435] and itemset [609], let's analyze their performance
based on the given metrics.  **Support:** -...
Completed 117/121
Waiting 4.5s...

[118/121] Sending request to Groq (attempt 1)
✅ Response received
To compare the performance of itemset [41, 994] with itemset [39, 4084], we
will evaluate the following metrics:  1. **N...
Completed 118/121
```

```
Waiting 4.5s...

[119/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 291] satisfies the frequent itemset (FI)
criteria, we need to check if its support is g...
Completed 119/121
Waiting 4.5s...

[120/121] Sending request to Groq (attempt 1)
✅ Response received
To determine if the itemset [39, 48, 5439] satisfies the frequent itemset (FI)
criteria, we need to check if its support...
Completed 120/121
💾 Checkpoint saved (120/121)
Waiting 4.5s...

[121/121] Sending request to Groq (attempt 1)
✅ Response received
To determine why the itemset [48, 41, 2996] qualifies as a frequent itemset
(FI), let's analyze its metrics:  1. **Suppo...
Completed 121/121
💾 Checkpoint saved (121/121)

========================================
All Groq responses completed.
Start evaluation...
========================================


========================================
Computing evaluation metrics...

Computing BERTScore...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a
HF_TOKEN to enable higher rate limits and faster downloads.
Loading weights: 100%|██████████| 389/389 [00:00<00:00, 8571.86it/s]
[transformers] RobertaModel LOAD REPORT from: roberta-large
Key                        | Status     |
---------------------------+------------+-
lm_head.dense.weight       | UNEXPECTED |
lm_head.layer_norm.weight  | UNEXPECTED |
lm_head.bias               | UNEXPECTED |
lm_head.layer_norm.bias    | UNEXPECTED |
lm_head.dense.bias         | UNEXPECTED |
pooler.dense.weight        | MISSING    |
pooler.dense.bias          | MISSING    |

Notes:
- UNEXPECTED:   can be ignored when loading from different task/architecture;
not ok if you expect identical arch.
- MISSING:      those params were newly initialized because missing from the
checkpoint. Consider training on your downstream task.
calculating scores...
computing bert embedding.
100%|██████████| 4/4 [00:18<00:00,  4.71s/it]
computing greedy matching.
100%|██████████| 2/2 [00:00<00:00, 32.47it/s]
```

```
done in 18.90 seconds, 6.40 sentences/sec

Computing ROUGE-L...

Saving CSV...

========================================
Average BERTScore
----------------------------------------
Precision : 0.8343
Recall    : 0.8610
F1        : 0.8472

========================================
Average ROUGE-L
----------------------------------------
Precision : 0.1561
Recall    : 0.4301
F1        : 0.2186

========================================
Evaluation completed.
Output CSV : ./Training_Data/Baseline_Groq_Llama-3.1-8B_predictions.csv
========================================
```

** Đọc kết quả file CSV **

In [8]:

```python
import pandas as pd
import re

# ============================================================
# Extract numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all integers and decimal numbers from text.
    """
    return re.findall(r"\d+(?:\.\d+)?", text)


# ============================================================
# Numerical Faithfulness
# ============================================================

def numerical_faithfulness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)
    prediction_numbers = extract_numbers(prediction_text)

    # Prediction không sinh số
    if len(prediction_numbers) == 0:
        return {
            "InputNumbers": input_numbers,
            "PredictionNumbers": prediction_numbers,
            "HallucinatedNumbers": [],
            "CorrectNumbers": 0,
            "PredictionCount": 0,
            "NumericalFaithfulness": 1.0
        }

    correct = 0
```

```
    hallucinated = []

    for num in prediction_numbers:

        if num in input_numbers:
            correct += 1
        else:
            hallucinated.append(num)

    score = correct / len(prediction_numbers)

    return {
        "InputNumbers": input_numbers,
        "PredictionNumbers": prediction_numbers,
        "HallucinatedNumbers": hallucinated,
        "CorrectNumbers": correct,
        "PredictionCount": len(prediction_numbers),
        "NumericalFaithfulness": score
    }


# ============================================================
# Evaluation
# ============================================================


df = pd.read_csv(OUTPUT_CSV, encoding="utf-8")

results = []

for sample, prediction in df[["input",
"prediction"]].itertuples(index=False):

    result = numerical_faithfulness(
        sample,
        prediction
    )

    results.append(result)

result_df = pd.DataFrame(results)

print("=" * 60)
print("Average Numerical Faithfulness:",
      result_df["NumericalFaithfulness"].mean())
print("=" * 60)

result_df.to_csv(
    f"NumericalFaithfulness_{nmodel}.csv",
    index=False,
    encoding="utf-8-sig"
)
```

```
============================================================
Average Numerical Faithfulness: 0.815342422389299
============================================================
```

** Numerical Faithfulness **

** Explanation Completeness **

In [9]:
```
import re
```

```
# ============================================================
# Keywords
# ============================================================

SUPPOR_KEYWORDS = [
    "support",
    "supports",
    "support threshold",
    "support exceeds",
    "purchased frequently",
    "frequently occurring",
    "frequent itemset"
]

AVG_LENGTH_KEYWORDS = [
    "average support-transaction length",
    "average transaction length",
    "longer transaction",
    "shorter transaction",
    "longer transactions",
    "shorter transactions",
    "average length"
]

FI_SCORE_KEYWORDS = [

    "fi score",

    "score",

    "overall performance",

    "better performance",

    "higher score",

    "lower score",

    "overall score",

    "business impact",

    "overall impact"

]


# ============================================================
# Utility
# ============================================================

def contains_keywords(text, keywords):

    text = text.lower()

    return any(keyword.lower() in text for keyword in keywords)


# ============================================================
# Support
# ============================================================
```

```
def evaluate_support(prediction):

    return int(
        contains_keywords(
            prediction,
            SUPPOR_KEYWORDS
        )
    )


# ============================================================
# Average Support Length
# ============================================================

def evaluate_average_length(prediction):

    return int(
        contains_keywords(
            prediction,
            AVG_LENGTH_KEYWORDS
        )
    )


# ============================================================
# FI Score
# ============================================================

def evaluate_fi_score(prediction):

    return int(
        contains_keywords(
            prediction,
            FI_SCORE_KEYWORDS
        )
    )


# ============================================================
# Overall
# ============================================================

def explanation_completeness(prediction):

    support = evaluate_support(prediction)

    avg = evaluate_average_length(prediction)

    fi = evaluate_fi_score(prediction)

    return {
        "support": support,
        "average_length": avg,
        "fi_score": fi,
        "ExplanationCompleteness":
            (support + avg + fi) / 3
    }


rows = []
```

```
for prediction in df["prediction"]:

    rows.append(
        explanation_completeness(
            prediction
        )
    )

result_df = pd.DataFrame(rows)

print("=" * 50)
print(result_df.mean(numeric_only=True))

result_df.to_csv(
    f"./Training_Data/ExplanationCompleteness_{nmodel}.csv",
    index=False
)
```

```
==================================================
support                     1.000000
average_length              0.942149
fi_score                    0.892562
ExplanationCompleteness     0.944904
dtype: float64
```

** Numerical Correctness **

In [10]:

```
import re
import pandas as pd

# ============================================================
# CONFIG
# ============================================================

TOLERANCE = 0.1

# ============================================================
# Remove Itemset IDs
# ============================================================

def remove_itemsets(text):
    """
    Remove itemset IDs.

    Example:
        [39, 41, 1799]
        [6009]
    """

    if text is None:
        return ""

    return re.sub(r"\[[^\]]*\]", "", text)


# ============================================================
# Extract Numbers
# ============================================================

def extract_numbers(text):
    """
    Extract all numerical values.
```

```
    """

    text = remove_itemsets(text)

    pattern = r"-?\d+(?:\.\d+)?"

    numbers = re.findall(pattern, text)

    return [float(x) for x in numbers]


# ============================================================
# Direct Match
# ============================================================

def is_direct_match(pred, input_numbers):

    for x in input_numbers:

        if abs(pred - x) <= TOLERANCE:
            return True

    return False


# ============================================================
# Division Match
# ============================================================

def is_division_match(pred, input_numbers):

    n = len(input_numbers)

    for i in range(n):

        for j in range(n):

            if i == j:
                continue

            a = input_numbers[i]
            b = input_numbers[j]

            # if abs(b) <= TOLERANCE:
            #     continue

            ratio = a / b

            if abs(pred - ratio) <= TOLERANCE:
                return True

    return False


# ============================================================
# Numerical Correctness
# ============================================================

def numerical_correctness(input_text, prediction_text):

    input_numbers = extract_numbers(input_text)
```

```
    pred_numbers = extract_numbers(prediction_text)

    direct_match = 0
    division_match = 0
    incorrect = 0

    for pred in pred_numbers:

        # 1. Có trực tiếp trong input
        if is_direct_match(pred, input_numbers):

            direct_match += 1

        # 2. Không có nhưng bằng phép chia
        elif is_division_match(pred, input_numbers):

            division_match += 1

        # 3. Sai hoàn toàn
        else:

            incorrect += 1

    correct = direct_match + division_match

    if len(pred_numbers) == 0:
        score = 0
    else:
        score = correct / len(pred_numbers)

    return {

        "input_numbers": len(input_numbers),

        "predicted_numbers": len(pred_numbers),

        "direct_match": direct_match,

        "division_match": division_match,

        "correct": correct,

        "incorrect": incorrect,

        "NumericalCorrectness": score
    }


# ============================================================
# Evaluate
# ============================================================

rows = []

for input_text, prediction in df[["input",
"prediction"]].itertuples(index=False):

    rows.append(
        numerical_correctness(
            input_text,
```

```
            prediction
        )
    )

df = pd.DataFrame(rows)

print(df)

print("="*70)
print("Average")
print("="*70)
print(df.mean(numeric_only=True))

print()

print("="*70)
print("Percentage")
print("="*70)

print(f"Average numbers in Input      : {df['input_numbers'].mean():.2f}")
print(f"Average numbers in Prediction :
{df['predicted_numbers'].mean():.2f}")
print(f"Direct Match                  : {df['direct_match'].mean():.2f}")
print(f"Division Match                : {df['division_match'].mean():.2f}")
print(f"Correct Numbers               : {df['correct'].mean():.2f}")
print(f"Incorrect Numbers             : {df['incorrect'].mean():.2f}")
print(f"Numerical Correctness         :
{df['NumericalCorrectness'].mean()*100:.2f}%")

df.to_csv(
    f"./Training_Data/NumericalCorrectness_{nmodel}.csv",
    index=False
)

print("\nSaved to NumericalCorrectness.csv")
```

```
     input_numbers  predicted_numbers  direct_match  division_match  correct  \
0               16                 15            10               3       13
1                8                 10             7               0        7
2               16                 15            10               3       13
3               16                 14            14               0       14
4                8                  4             4               0        4
..             ...                ...           ...             ...      ...
116             17                 11            11               0       11
117             16                  9             6               2        8
118              8                  7             7               0        7
119              8                  8             7               0        7
120              8                 16            10               1       11

     incorrect  NumericalCorrectness
0            2              0.866667
1            3              0.700000
2            2              0.866667
3            0              1.000000
4            0              1.000000
..         ...                   ...
116          0              1.000000
117          1              0.888889
118          0              1.000000
119          1              0.875000
120          5              0.687500
```

```
[121 rows x 7 columns]
================================================================================
Average
================================================================================
input_numbers           12.181818
predicted_numbers       13.132231
direct_match             9.776860
division_match           1.685950
correct                 11.462810
incorrect                1.669421
NumericalCorrectness     0.891902
dtype: float64

================================================================================
Percentage
================================================================================
Average numbers in Input      : 12.18
Average numbers in Prediction : 13.13
Direct Match                  : 9.78
Division Match                : 1.69
Correct Numbers               : 11.46
Incorrect Numbers             : 1.67
Numerical Correctness         : 89.19%

Saved to NumericalCorrectness.csv
```

In [ ]: